МИНСПОРТ РОССИИ

ФСП федерация спортивного программирования России

КУБОК ФЕДЕРАЦИИ 2026

РАЙТАП · FORENSICS

ПЕРВАЯ ЗАПИСЬ

FORENSICS

19 СЕНТЯБРЯ 2026
ОНЛАЙН · CTF TASK-BASED

FORENSICS
КАТЕГОРИЯ

37
СЛАЙДОВ В РАЗБОРЕ

01

ИСХОДНЫЕ ДАННЫЕ

ПЕРВАЯ ЗАПИСЬ
FORENSICS

Описание:

Ноутбук из бухгалтерии уехал в утилизацию, но перед этим с него сняли полный дамп SPI-флеша и образ EFI-раздела.

Пользователь клянётся, что ставил только "официальное" обновление BIOS с портала вендора. Инцидент-репорт тем временем упоминает, что после апдейта на машине один раз заглянули в ledger.cfg, которого там давно нет. Проверьте сами.

ФАЙЛЫ ЗАДАНИЯ:

```
spi_dump.bin
esp.img
vendor_update.zip
```

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

Распаковываем архив с обновлением, внутри `OVMF_CODE.fd (3653632)` и `OVMF_VARS.fd (540672)`. Сложив размеры полученных файлов, выходит ровно размер дампа SPI. То есть "ноутбук" - это QEMU/OVMF-виртуалка: SPI = CODE + VARS, а обновление кладет рядом свежие `OVMF_CODE.fd` / `OVMF_VARS.fd`. Это даёт нам эталон чистой прошивки.

Побайтово дамп SPI не равен конкатенации файлов из архива, VARS-область расходится на ~13 КБ (запись NVRAM при загрузке), а CODE - на сотни КБ. Но это не значит, что код другой, ведь он сжат LZMA внутри FVMAIN_COMPACT, поэтому сравнивать нужно на уровне FFS-файлов.

На ESP файловая система FAT, TSK можно найти удаленную запись:

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
$ fls -r esp.img
---
r/r 3:  FUNDFUND    (Volume Label Entry)
d/d 4:  EFI
+ d/d 517:  BOOT
++ r/r 581: BOOTX64.EFI
r/r 5:  ledger.cfg
...
$ icat esp.img 5
---
# ledger record - fund 1147, opis 1, delo 9
[esp]
bootorder=hd,internal-shell
console=serial,115200
recovery_pin=1147-0091
note=digitization batch 9: originals sealed in the depository
first_record=REDACTED-TRANSFERRED-TO-MAIN-VAULT
```

`first_record` отредактирован, значит, настоящий секрет уже не в ESP, а где-то в прошивке/NVRAM.

Разберем структуру образа. Сначала находим сигнатуры `_FVH` (смещение 0x28 внутри заголовка FV, значит база FV = смещение - 0x28):

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
grep -abo '_FVH' spi_dump.bin
---
40:_FVH        > FV @ 0x000000
3440680:_FVH   > FV @ 0x348000
3653672:_FVH   > FV @ 0x37C000   (остальные совпадения — мусор внутри LZMA)
```

Заголовок первого FV (и его расшифровка по `EFI_FIRMWARE_VOLUME_HEADER`):

```
00000000: 00*16                          ZeroVector
00000010: 78 e5 8c 8c 3d 8a 1c 4f ...    FileSystemGuid =
                                         {8C8CE578-8A3D-4F1C-9935-896185C32DD3}
                                         = EFI_FIRMWARE_FILE_SYSTEM2_GUID (FFS2)
00000020: 00 80 34 00 00 00 00 00        FvLength       = 0x348000
00000028: 5f 46 56 48                    Signature '_FVH'
00000030: 48 00 f3 62                    HeaderLength=0x48, Checksum
00000034: 60 00 00 02                    ExtHeaderOffset=0x60, Revision=2
00000038: 48 03 00 00 00 10 00 00        BlockMap: 0x348 блоков по 0x1000
00000060: 17 5e db 48 7c 70 2d 47 ...    ExtHeader: FvNameGuid
                                         {48DB5E17-707C-472D-91CD-1613E7EF51B0}
```

`FvNameGuid` из исходников OVMF опознаётся как `FVMAIN_COMPACT`. Подобным образом читаем остальные два:

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
import uuid; d=open('spi_dump.bin','rb').read()
print(uuid.UUID(bytes_le=d[0x348010:0x348020]))  # {763BED0D-DE9F-48F5-81F1-3E90E1B1A015} = SECFV
print(uuid.UUID(bytes_le=d[0x37C010:0x37C020]))  # {FFF12B8D-7696-4C8B-A985-2747075B4F50}
                                                 # = EFI_SYSTEM_NV_DATA_FV_GUID (хранилище переменных)
```

Итого карта дампа: `[0x000000..0x348000)` FVMAIN_COMPACT, `[0x348000..0x37C000)` SECFV (там находится SecMain `{DF1CCEF6-…}`, 45 КБ), `[0x37C000..0x400000)` NVRAM. Та же раскладка у `OVMF_CODE.fd` из архива, сравнивать будем содержимое.

В FFS2 каждый файл начинается с заголовка 24 байт: имя GUID [16], проверка целостности [2], тип, атрибуты, размер [3], состояние. После ext-header файлы в FVMAIN_COMPACT начинаются с 0x78 и их ровно один:

```
import struct, uuid
d=open('update/OVMF_CODE.fd','rb').read()
pos=0x78                                  # (ExtHeaderOffset+ExtSize+7)&~7
g=uuid.UUID(bytes_le=d[pos:pos+16])
size=d[pos+20]|(d[pos+21]<<8)|(d[pos+22]<<16)     # u24 Size
print(g, 'type=%#x'%d[pos+18], 'size=', size, 'state=%#x'%d[pos+23])
---
9e21fd93-9c72-4c15-8c4b-e77f1db2d792 type=0xb size= 1022373 state=0xf8
```

Тип `0x0B` = `EFI_FV_FILETYPE_FIRMWARE_VOLUME_IMAGE`, состояние `0xF8` - валиден. Внутри тела файла одна секция типа `0x02` (GUID_DEFINED), её заголовок несёт GUID декомпрессора и смещение данных:

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
import struct, lzma
def unpack_lzma_fvmain(src, dst):
    d=open(src,'rb').read()
    pos=0x78                                    # единственный FFS-файл после ext-header
    size=d[pos+20]|(d[pos+21]<<8)|(d[pos+22]<<16)
    body=d[pos+24:pos+size]                     # секции файла
    ssize=body[0]|(body[1]<<8)|(body[2]<<16)    # размер GUID_DEFINED-секции
    assert body[3]==0x02                        # 0x02 = EFI_SECTION_GUID_DEFINED
    # body[4:20] = ee4e5898-3914-4259-9d6e-dc7bd79403cf (LZMA_CUSTOM_DECOMPRESS)
    doff=struct.unpack('<H',body[20:22])[0]     # 0x18
    payload=body[doff:ssize]
    props=payload[:5]; usize=struct.unpack('<Q',payload[5:13])[0]
    print(src, 'props=%s usize=%d'%(props.hex(), usize))
    dec=lzma.LZMADecompressor(format=lzma.FORMAT_RAW,
        filters=[lzma._decode_filter_properties(lzma.FILTER_LZMA1, props)])
    out=dec.decompress(payload[13:])
    assert len(out)==usize
    open(dst,'wb').write(out)

unpack_lzma_fvmain('update/OVMF_CODE.fd', 'update_fvmain.bin')
unpack_lzma_fvmain('spi_dump.bin','spi_fvmain.bin')
---
update/OVMF_CODE.fd props=5d00000001 usize=16122000
spi_dump.bin props=5d00000001 usize=16122000
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

Форма пейлоад EFI LZMA, то есть 5 байт props + u64 несжатого размера + сырой LZMA1-поток. Распакованный поток это снова список секций:

```
import sys; sys.path.insert(0,'.')
from helpers.fvscan import parse_fv    # сурсы хелпер скриптов см. ниже
d=open('update_fvmain.bin','rb').read()
pos=0
while pos+4⩽len(d):
    ssize=d[pos]|(d[pos+1]<<8)|(d[pos+2]<<16); stype=d[pos+3]
    if ssize<4 or pos+ssize>len(d): break
    print('section @%#010x type=%#04x size=%d'%(pos,stype,ssize))
    if stype==0x17:
        res=[]; parse_fv(d[pos+4:pos+ssize],0,res,'')
        n=sum(1 for r in res if r[0]=='FILE'); print('           files:',n)
    pos=(pos+ssize+3)&~3
---
section @0x00000000 type=0x19 size=124      (паддинг)
section @0x0000007c type=0x17 size=917508   > PEIFV
           files: 1
section @0x000e0080 type=0x19 size=12       (паддинг)
section @0x000e008c type=0x17 size=15204356 > DXEFV
           files: 105
```

Внутри каждого FV_IMAGE обычный FFS, значит обходим файлы (state == 0xff/ff*16 - конец цепочки), читаем их секции; имя модуля получаем из UI-секции (0x15, UTF-16LE).

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
import sys; sys.path.insert(0,'.')
from helpers.deep import walk_sections
out=[]
walk_sections(open('update_fvmain.bin','rb').read(),'root',out)
files=[r for r in out if r[0].startswith('{')]
print('total:',len(files))
for g,t,info in files:
    print(g,t,info)
---
total: 106
{1B45CC0A-156A-428A-AF62-49864DA0E6E6} FREEFORM size=20
{FC510EE7-FFDC-11D4-BD41-0080C73C8881} FREEFORM size=84
{D6A2CB7F-6A18-4E2F-B43B-9920A733700A} COMBINED_PEIM_DRIVER size=138278 DxeCore
{D93CE3D8-A7EB-4730-8C8E-CC466A9ECC3C} SMM size=16514 ReportStatusCodeRouterRuntimeDxe
{6C2004EF-4E0E-4BE4-B14C-340EB4AA5891} SMM size=12426 StatusCodeHandlerRuntimeDxe
{80CF7257-87AB-47F9-A3FE-D50B76D89541} SMM size=10258 PcdDxe
{B601F8C4-43B7-4784-95B1-F4226CB40CEE} SMM size=16470 RuntimeDxe
{F80697E9-7FD6-4665-8646-88E33EF71DFC} SMM size=9694 SecurityStubDxe
{13AC6DD0-73D0-11D4-B06B-00AA00BD6DE7} SMM size=15934 EbcDxe
{A19B1FE7-C1BC-49F8-875F-54A5D542443F} SMM size=4482 CpuIo2Dxe
{1A1E4886-9517-440E-9FDE-3BE44CEE2136} SMM size=91298 CpuDxe
{6490F1C5-EBCC-4665-8892-0075B9BB49B7} SMM size=70178 CpuDxe
...
```

Сравниваем множества GUID:

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
from helpers.deep import walk_sections
def inv(p):
    out=[]; walk_sections(open(p,'rb').read(),'root',out)
    return [r for r in out if r[0].startswith('{')]
a,b=inv('update_fvmain.bin'),inv('spi_fvmain.bin')
ga,gb={r[0] for r in a},{r[0] for r in b}
print('update',len(a),'spi',len(b))
for g in sorted(gb-ga): print('UNIQUE SPI:',g,[r for r in b if r[0]==g])
for g in sorted(ga-gb): print('UNIQUE UPD:',g)
---
update 106 spi 109
ONLY IN SPI: {3A91C7E2-5D64-4F0B-8A19-C7E2D5B40F36} [('{3A91C7E2-5D64-4F0B-8A19-C7E2D5B40F36}', 'SMM', 'siz
e=1482 BoardSkuDxe')]
ONLY IN SPI: {6F3D9A41-2C8B-4E17-B05A-93D7C4E21F88} [('{6F3D9A41-2C8B-4E17-B05A-93D7C4E21F88}', 'SMM', 'siz
e=5958 LedgerDxe')]
ONLY IN SPI: {B14E8D5F-6A23-4C97-9F02-1D5E8B3A6C44} [('{B14E8D5F-6A23-4C97-9F02-1D5E8B3A6C44}', 'SMM', 'siz
e=1486 ChassisTagDxe')]
```

Реальная прошивка несёт три вшитых DXE-драйвера, которых в апдейте нет. Выгружаем их PE32 на диск:

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
$ python extract.py ../spi_fvmain.bin ../spi_modules
---
...
../spi_modules/6F3D9A41-2C8B-4E17-B05A-93D7C4E21F88_21.pe 5696
../spi_modules/3A91C7E2-5D64-4F0B-8A19-C7E2D5B40F36_22.pe 1216
../spi_modules/B14E8D5F-6A23-4C97-9F02-1D5E8B3A6C44_23.pe 1216
...
```

Интересует нас самый большой - LedgerDxe (PE32+ boot-service driver). В .data лежат два опознаваемых GUID (сверяемся с репозиторием edk2):

- `0x104C0`: `{7CE88FB3-4BD7-4679-87A8-A8D8DEE50D2B}` = gEfiEventReadyToBootGuid (`EFI_EVENT_GROUP_READY_TO_BOOT`);
- `0x104D0`: `{964E5B22-6459-11D2-8E39-00A0C969723B}` = gEfiSimpleFileSystemProtocolGuid.

Декомпилируем entry функцию:

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
void entry0(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE *SystemTable)
{
    gBS = SystemTable→BootServices;      // [rdx+0x60] → сохраняется в 0x10500/0x11500
    gRT = SystemTable→RuntimeServices;   // [rdx+0x58] → сохраняется в 0x104f0
    gST = SystemTable;

    // CreateEventEx(EVT_NOTIFY_SIGNAL, TPL_CALLBACK, notify_10240, NULL,
    //               &gEfiEventReadyToBootGuid, &Event)
    gBS→CreateEventEx(0x200, 8, fcn_00010240, NULL, &GUID_ReadyToBoot /*0x104c0*/, &ev);
}
```

То есть вся полезная нагрузка исполняется по событию ReadyToBoot - его BDS сигнализирует (`EfiSignalEvent-ReadyToBoot`) перед каждым запуском boot-опции, в этот момент диспетчер DXE уже отработал, boot сервисы ещё живы, файловые протоколы и `SetVariable(NV|BS)` полностью работоспособны. Это удобная точка, код отрабатывает ровно тогда, когда пользователь пытается загрузить систему.

Разберем колбэк ReadyToBoot, где читается и расшифровывается `ledger.cfg` :

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
void notify_end_of_dxe(void)
{
    EFI_SIMPLE_FILE_SYSTEM_PROTOCOL *sfs;
    EFI_FILE_PROTOCOL *root, *file;
    EFI_STATUS st;
    UINTN size;

    // 1) найти файловую систему (ESP)
    if ((st = gBS→LocateProtocol(&gEfiSimpleFileSystemProtocolGuid/*0x104d0*/,
                                 NULL, &sfs)) < 0) return;
    // 2) открыть корень и \ledger.cfg (имя UTF-16LE по адресу 0x10456)
    sfs→OpenVolume(sfs, &root);                                    // call [x+0x08]
    if (root→Open(root, &file, L"ledger.cfg", EFI_FILE_MODE_READ, 0) < 0) {
        root→Close(root);
        return;
    }
    // 3) прочитать весь файл В буфер драйвера .data:0x1050c (лимит 0xFF4)
    size = 0xff4;
    st = file→Read(file, &size, (VOID*)0x1050c);                   // call [x+0x20]
    file→Close(file);
    root→Close(root);
    if (st < 0 || size == 0) return;

    // 4) seed = key[8..15] как big-endian u64
    //    (упаковка 16 байт ключа сдвигами вымывает стартовую константу
    //     0x9E3779B97F4A7C15 за те же 16 итераций)
    uint64_t seed = 0x9e3779b97f4a7c15;
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
for (int i = 0; i < 16; i++)
    seed = (seed << 8) | key_blob[i];          // key_blob @ 0x10490
// seed == 0x1FA945CC73E608BB

// 5) инициализация состояния гаммы: множим длину файла, xor seed,
//    затем один лишний xorshift-27 перед циклом
uint64_t st64 = (size * 0xd1b54a32d192ed03) ^ seed;
st64 ^= st64 >> 27;

// 6) генерация гаммы и XOR in-place (финализатор splitmix64-подобный)
for (UINTN i = 0; i < size; i++) {
    uint64_t t  = st64 ^ (st64 >> 12);         // 0x35d..0x364
    t           = t   ^ (t   << 25);           // 0x367..0x36e
    st64        = t   ^ (t   >> 27);           // 0x371..0x378
    uint8_t kb  = (st64 * 0x2545f4914f6cdd1d) >> 56;   // старший байт
    ((uint8_t*)0x1050c)[i] ^= kb;              // 0x386
}

// 7) заголовок записи и эксфильтрация в NVRAM
*(uint32_t*)0x10500 = 0x44555354;              // сигнатура 'TSUD'
*(uint32_t*)0x10504 = (uint32_t)size;          // длина полезной нагрузки
*(uint32_t*)0x10508 = 0x126;                   // служебное поле

// SetVariable(L"Ledger1147", {4E2F1A33-8B5D-4C44-9E1B-63F0A2C7D901},
//             NV|BS, size+12, header+plaintext)
gRT->SetVariable(L"Ledger1147"/*0x1046c*/, &LedgerGuid/*0x104a0*/, 3,
                 size + 12, (VOID*)0x10500);   // call [gRT+0x58]
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
}
```

Ключевой блок данных `.data`:

```
0x10490: 5A C3 11 7E 94 2B D8 60 1F A9 45 CC 73 E6 08 BB   ; key_blob[16]
0x104A0: 33 1A 2F 4E 5D 8B 44 4C 9E 1B 63 F0 A2 C7 D9 01   ; VendorGuid переменной
0x10456: L"ledger.cfg"        0x1046C: L"Ledger1147"
```

Итого: при каждом старте прошивка читает `ledger.cfg` с ESP, расшифровывает самописной гаммой и складывает результат в UEFI-переменную `{4E2F1A33-…}:Ledger1147` - главное хранилище = NVRAM во флеше.

Два остальных драйвера - декои, устроены одинаково просто, тот же `CreateEventEx` на ReadyToBoot, а колбэк одна строка:

```
// BoardSkuDxe (fcn.00010240), VendorGuid {77AA1B05-3E92-4D6C-A411-28D95FC03B7E}
gRT→SetVariable(L"BoardSku", &guid, NV|BS, 0x17, "SKU-BKUP01-REV2-BATCH07");

// ChassisTagDxe (fcn.00010240), VendorGuid {2E9D5C18-7B40-49A3-86D2-51E09A6CF412}
gRT→SetVariable(L"ChassisTag", &guid, NV|BS, 0x0a, "CHT-0091-A");
```

Получим флаг. Переменная живёт в VARS-области дампа (0x37C000…0×400000). Ищем VendorGuid:

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```python
import uuid
data = open('spi_dump.bin','rb').read()
g = uuid.UUID('4e2f1a33-8b5d-4c44-9e1b-63f0a2c7d901').bytes_le
i  = data.find(g)        # 0x37e6c8 (старая ревизия) и 0x37f604 (новая)

ns = int.from_bytes(data[i-8:i-4],'little')   # NameSize  = 22 (L"Ledger1147")
ds = int.from_bytes(data[i-4:i],  'little')   # DataSize
blob = data[i+16+ns : i+16+ns+ds]             # 'TSUD' | len | 0x126 | payload
ct   = blob[12:]
```

Находим две копии переменной, variable store хранит устаревшие ревизии (состояние у старой `0x3c` - удалена, у новой `0x3f` - валидна):

РЕВИЗИЯ	STATE	DATASIZE	PAYLOAD LEN	СОДЕРЖИМОЕ ПОСЛЕ РАСШИФРОВКИ
0×37E6C8	`0x3c` (deleted)	244	232	оригинал с флагом
0×37F604	`0x3f` (valid)	245	233	отредактированный текст с `REDACTED-…`

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```python
M = (1<<64)-1
KEY = bytes.fromhex('5ac3117e942bd8601fa945cc73e608bb')   # key_blob @ 0x10490

def crypt(buf):
    seed = int.from_bytes(KEY[8:16], 'big')               # 0x1FA945CC73E608BB
    st = ((0xD1B54A32D192ED03 * len(buf)) & M) ^ seed
    st ^= st >> 27
    out = bytearray()
    for b in buf:
        t  = st ^ (st >> 12)
        t  = t ^ ((t << 25) & M)
        st = t ^ (t >> 27)
        out.append(b ^ (((st * 0x2545F4914F6CDD1D) & M) >> 56))
    return bytes(out)

print(crypt(ct).decode())
```

Получаем на выходе:

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
python decrypt.py
---
# ledger record - fund 1147, opis 1, delo 9
[esp]
bootorder=hd,internal-shell
console=serial,115200
recovery_pin=1147-0091
note=digitization batch 9: originals sealed in the depository
first_record=flag{1147_f1rst_r3c0rd_w4s_f0und}
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

ПЕРВАЯ ЗАПИСЬ
FORENSICS

Хелпер скрипты

```python
# fvscan.py
import struct, uuid, lzma, sys, hashlib, collections

FVH = b'_FVH'
SEC_TYPES = {0x01:'COMPRESSION',0x02:'GUID_DEFINED',0x10:'PE32',0x11:'PIC',0x12:'TE',
             0x13:'DXE_DEPEX',0x14:'VERSION',0x15:'UI',0x16:'APPLICABILITY',
             0x17:'FV_IMAGE',0x18:'FREEFORM',0x19:'RAW',0x1B:'PEI_DEPEX',0x1C:'SMM_DEPEX'}
FILE_TYPES = {0x01:'RAW',0x02:'FREEFORM',0x03:'CORE_STAGED',0x04:'DRIVER',0x05:'COMBINED_PEIM_DRIVER',
              0x06:'APPLICATION',0x07:'SMM',0x08:'FIRMWARE_VOLUME_IMAGE',0x09:'COMBINED_SMM_DXE',
              0x0A:'SMM_CORE',0xB0:'OEM_MIN',0xC0:'FFS_PAD'}
LZMA_GUID = uuid.UUID('ee4e5898-3914-4259-9d6e-dc7bd79403cf')
LZMAF86_GUID = uuid.UUID('d42ae6bd-1352-4bfb-909a-ca72a6eae889')

def u24(b): return b[0] | (b[1]<<8) | (b[2]<<16)

def lzma_dec(data):
    # EFI custom: 5-byte props + 8-byte size, raw LZMA1
    props = data[:5]; usize = struct.unpack('<Q', data[5:13])[0]
    d = lzma.LZMADecompressor(format=lzma.FORMAT_RAW,
        filters=[lzma._decode_filter_properties(lzma.FILTER_LZMA1, props)])
    return d.decompress(data[13:], max_length=usize)

def parse_fv(data, off, results, path, depth=0):
    if data[off+40:off+44] != FVH: return None
    fvlen = struct.unpack('<Q', data[off+32:off+40])[0]
```

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
    hdrlen = struct.unpack('<H', data[off+48:off+50])[0]
    exthdr = struct.unpack('<H', data[off+52:off+54])[0]
    start = off + hdrlen
    if exthdr:
        extsize = struct.unpack('<I', data[off+exthdr+16:off+exthdr+20])[0]
        start = off + ((exthdr + extsize + 7) & ~7)
    results.append(('FV', path, hex(off), 'len=%#x'%fvlen))
    pos = start
    end = off + fvlen
    while pos + 24 ⩽ min(end, len(data)):
        name = data[pos:pos+16]
        state = data[pos+23]
        if name == b'\xff'*16 or state == 0xFF: break
        ftype = data[pos+18]; fattr = data[pos+19]
        size = u24(data[pos+20:pos+23])
        hdrsz = 24
        if size == 0xFFFFFF:
            size = struct.unpack('<I', data[pos+24:pos+28])[0]; hdrsz = 28
        if size < hdrsz: break
        g = '{%s}' % str(uuid.UUID(bytes_le=name)).upper()
        body = data[pos+hdrsz:pos+size]
        results.append(('FILE', g, FILE_TYPES.get(ftype,hex(ftype)), 'size=%d'%len(body), path))
        parse_sections(body, g, results, depth+1)
        pos = (pos + size + 7) & ~7
    return off + fvlen

def parse_sections(data, owner, results, depth, prefix=''):
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
pos = 0
while pos + 4 ⩽ len(data):
    ssize = u24(data[pos:pos+3])
    stype = data[pos+3]
    if ssize < 4 or pos+ssize > len(data):
        if all(b==0xff for b in data[pos:]): return
        ssize = min(len(data)-pos, max(ssize,4)) if ssize⩾4 else (len(data)-pos)
    sec = data[pos+4:pos+ssize]
    tname = SEC_TYPES.get(stype, hex(stype))
    if stype in (0x01,) and not prefix:
        dec = None
        try: dec = lzma_dec(sec)
        except Exception: pass
        if dec is None:
            try: dec = lzma.decompress(sec) # plain
            except Exception: dec = None
        if dec is not None:
            results.append(('SEC', tname+'(decompressed)', '%d→%d'%(len(sec),len(dec)), '', owner))
            parse_sections(dec, owner, results, depth, prefix='c/')
            pos += (ssize+3)&~3
            continue
    if stype == 0x02 and not prefix:
        sg = uuid.UUID(bytes_le=sec[:16])
        doff = struct.unpack('<H', sec[16:18])[0]
        sattr = struct.unpack('<H', sec[18:20])[0]
        payload = sec[doff:]
        if sg in (LZMA_GUID, LZMAF86_GUID):
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
            try:
                dec = lzma_dec(payload)
                if sg == LZMAF86_GUID:
                    dec = dec  # BCJ x86 filter applied? keep raw attempt
                results.append(('SEC', 'GUID:LZMA', '%d→%d'%(len(payload),len(dec)), str(sg), owner))
                parse_sections(dec, owner, results, depth, prefix='c/')
                pos += (ssize+3)&~3
                continue
            except Exception as e:
                pass
        label = ''
        if stype == 0x15:
            try: label = sec.decode('utf-16-le').rstrip('\x00')
            except Exception: pass
            results.append(('UI', label, '', '', owner))
        elif stype == 0x17:
            sub_off = parse_fv(sec, 0, [], '', depth+1)
            results.append(('SEC', 'FV_IMAGE', '%d bytes'%len(sec), '', owner))
            tmp=[]; parse_fv(sec, 0, tmp, '', depth+1)
            for r in tmp:
                if r[0]=='FILE':
                    results.append(('SUBFILE', r[1], r[2], r[3], owner+'/'+r[1]))
                    # recurse into sub-fv files' sections too
                    sub=[]
                    parse_subfv_files(sec, sub)
                    results.extend(sub)
        else:
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
            results.append(('SEC', tname, '%d bytes'%len(sec), label, owner))
        pos += (ssize+3)&~3

def parse_subfv_files(fvdata, out):
    res=[]
    parse_fv(fvdata, 0, res, '')
    # second pass to get sections of each file
    if fvdata[40:44]≠FVH: return
    fvlen=struct.unpack('<Q',fvdata[32:40])[0]; hdrlen=struct.unpack('<H',fvdata[48:50])[0]
    exthdr=struct.unpack('<H',fvdata[52:54])[0]
    start=hdrlen
    if exthdr:
        extsize=struct.unpack('<I',fvdata[exthdr+16:exthdr+20])[0]
        start=(exthdr+extsize+7)&~7
    pos=start; end=fvlen
    while pos+24⩽min(end,len(fvdata)):
        name=fvdata[pos:pos+16]
        if name==b'\xff'*16 or fvdata[pos+23]==0xFF: break
        ftype=fvdata[pos+18]
        size=u24(fvdata[pos+20:pos+23])
        hdrsz=24
        if size==0xFFFFFF:
            size=struct.unpack('<I',fvdata[pos+24:pos+28])[0]; hdrsz=28
        if size<hdrsz: break
        g='{%s}'%str(uuid.UUID(bytes_le=name)).upper()
        body=fvdata[pos+hdrsz:pos+size]
        out.append(('SUBFILE_SEC', g, FILE_TYPES.get(ftype,hex(ftype)), '%d bytes'%len(body), ''))
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
        parse_sections(body, g, out, 0, prefix='')
        pos=(pos+size+7)&~7

def extract(path):
    data=open(path,'rb').read()
    results=[]
    pos=0
    while True:
        idx=data.find(FVH,pos)
        if idx<0: break
        off=idx-40
        nxt=parse_fv(data, off, results, path)
        pos=idx+4
    return results

if __name__=='__main__':
    for p in sys.argv[1:]:
        print('#'*20, p)
        seen=set()
        for r in extract(p):
            key=str(r)
            if key in seen: continue
            seen.add(key)
            print(*r, sep=' | ')
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```python
# deep.py
import sys, struct, uuid, lzma, hashlib, os

sys.path.insert(0, ".")
from helpers.fvscan import parse_fv


def gstr(g):
    return "{%s}" % str(uuid.UUID(bytes_le=bytes(g))).upper()


def u24(b):
    return b[0] | (b[1] << 8) | (b[2] << 16)


def lzma_dec(payload):
    props = payload[:5]
    usize = struct.unpack("<Q", payload[5:13])[0]
    d = lzma.LZMADecompressor(
        format=lzma.FORMAT_RAW,
        filters=[lzma._decode_filter_properties(lzma.FILTER_LZMA1, props)],
    )
    return d.decompress(payload[13:], max_length=usize)


def walk_sections(data, owner, out):
    pos = 0
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
    while pos + 4 ⩽ len(data):
        ssize = u24(data[pos : pos + 3])
        stype = data[pos + 3]
        if ssize < 4 or pos + ssize > len(data):
            break
        sec = data[pos + 4 : pos + ssize]
        if stype == 0x02:
            sg = uuid.UUID(bytes_le=sec[:16])
            if sg in (
                uuid.UUID("ee4e5898-3914-4259-9d6e-dc7bd79403cf"),
                uuid.UUID("d42ae6bd-1352-4bfb-909a-ca72a6eae889"),
            ):
                doff = struct.unpack("<H", sec[16:18])[0]
                try:
                    dec = lzma_dec(sec[doff:])
                    out.append(("LZMA", owner, "%d→%d" % (len(sec), len(dec))))
                    walk_sections(dec, owner, out)
                except Exception as e:
                    out.append(("LZMA_FAIL", owner, str(e)))
        elif stype == 0x17:
            sub = data[pos + 4 : pos + ssize]
            res = []
            parse_fv(sub, 0, res, owner)
            for i, r in enumerate(res):
                if r[0] == "FILE":
                    ui = None
                    for r2 in res[i + 1 :]:
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
                    if r2[0] == "FILE":
                        break
                    if r2[0] == "UI":
                        ui = r2[1]
                out.append((r[1], r[2], "%s %s" % (r[3], ui or "")))
            # recurse into sub-fv sections to catch nested compressed
            walk_files_in_fv(sub, out)
        elif stype == 0x01:
            try:
                dec = lzma_dec(sec) if sec[:5] != b"\x00\x00\x00\x00\x00" else None
                if dec:
                    out.append(("COMPRESSION", owner, "%d→%d" % (len(sec), len(dec))))
                    walk_sections(dec, owner, out)
            except Exception:
                pass
        pos = (pos + ssize + 3) & ~3


def walk_files_in_fv(fvdata, out):
    if fvdata[40:44] != b"_FVH":
        return
    fvlen = struct.unpack("<Q", fvdata[32:40])[0]
    hdrlen = struct.unpack("<H", fvdata[48:50])[0]
    exthdr = struct.unpack("<H", fvdata[52:54])[0]
    start = hdrlen
    if exthdr:
        extsize = struct.unpack("<I", fvdata[exthdr + 16 : exthdr + 20])[0]
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
        start = (exthdr + extsize + 7) & ~7
    pos = start
    end = min(fvlen, len(fvdata))
    while pos + 24 ⩽ end:
        name = fvdata[pos : pos + 16]
        st = fvdata[pos + 23]
        if name == b"\xff" * 16 or st == 0xFF:
            break
        size = u24(fvdata[pos + 20 : pos + 23])
        hdrsz = 24
        if size == 0xFFFFFF:
            size = struct.unpack("<I", fvdata[pos + 24 : pos + 28])[0]
            hdrsz = 28
        if size < hdrsz:
            break
        body = fvdata[pos + hdrsz : pos + size]
        walk_sections(body, gstr(name), out)
        pos = (pos + size + 7) & ~7


def scan(path):
    data = open(path, "rb").read()
    out = []
    seen = set()
    idx = 0
    fvs = []
    while True:
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
        i = data.find(b"_FVH", idx)
        if i < 0:
            break
        off = i - 40
        if off >= 0 and off not in seen:
            res = []
            try:
                parse_fv(data, off, res, path)
            except Exception as e:
                pass
            if res:
                seen.add(off)
                fvs.append(off)
                out.extend(res)
        idx = i + 4
    # now walk each top-level FV fully (files+sections incl. lzma)
    for off in fvs:
        walk_fv_at(data, off, out)
    return out


def walk_fv_at(data, off, out):
    if data[off + 40 : off + 44] != b"_FVH":
        return
    fvlen = struct.unpack("<Q", data[off + 32 : off + 40])[0]
    hdrlen = struct.unpack("<H", data[off + 48 : off + 50])[0]
    exthdr = struct.unpack("<H", data[off + 52 : off + 54])[0]
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
start = off + hdrlen
if exthdr and exthdr > hdrlen:
    extsize = struct.unpack("<I", data[off + exthdr + 16 : off + exthdr + 20])[0]
    endh = (exthdr + extsize + 7) & ~7
    start = off + endh
else:
    # no ext header: files begin right after header
    pass
pos = start
end = min(off + fvlen, len(data))
while pos + 24 ⩽ end:
    name = data[pos : pos + 16]
    st = data[pos + 23]
    if name == b"\xff" * 16 or st == 0xFF:
        break
    size = u24(data[pos + 20 : pos + 23])
    hdrsz = 24
    if size == 0xFFFFFF:
        size = struct.unpack("<I", data[pos + 24 : pos + 28])[0]
        hdrsz = 28
    if size < hdrsz:
        break
    g = gstr(name)
    body = data[pos + hdrsz : pos + size]
    out.append(("FILE", g, "%d bytes" % len(body), ""))
    walk_sections(body, g, out)
    pos = (pos + size + 7) & ~7
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
if __name__ == "__main__":
    res = scan(sys.argv[1])
    for r in res:
        print(*r, sep=" | ")
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```python
# extract.py
import sys, struct, uuid
sys.path.insert(0,'.')

def u24(b): return b[0]|(b[1]<<8)|(b[2]<<16)

def parse_sections_raw(data, out):
    pos=0
    while pos+4⩽len(data):
        ssize=u24(data[pos:pos+3]); stype=data[pos+3]
        if ssize<4 or pos+ssize>len(data): break
        out.append((stype,data[pos+4:pos+ssize]))
        if stype==0x17:
            sub=[]
            parse_fvs_raw(data[pos+4:pos+ssize],sub)
            out.extend(sub)
        pos=(pos+ssize+3)&~3
    return

import lzma
def parse_fvs_raw(fvdata,out):
    if fvdata[40:44]≠b'_FVH': return
    fvlen=struct.unpack('<Q',fvdata[32:40])[0]
    hdrlen=struct.unpack('<H',fvdata[48:50])[0]
    exthdr=struct.unpack('<H',fvdata[52:54])[0]
    start=hdrlen
    if exthdr and exthdr⩾hdrlen:
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
        extsize=struct.unpack('<I',fvdata[exthdr+16:exthdr+20])[0]
        start=(exthdr+extsize+7)&~7
    pos=start; end=min(fvlen,len(fvdata))
    while pos+24⩽end:
        name=fvdata[pos:pos+16]; st=fvdata[pos+23]
        if name==b'\xff'*16 or st==0xFF: break
        size=u24(fvdata[pos+20:pos+23]); hdrsz=24
        if size==0xFFFFFF:
            size=struct.unpack('<I',fvdata[pos+24:pos+28])[0]; hdrsz=28
        if size<hdrsz: break
        g='{%s}'%str(uuid.UUID(bytes_le=fvdata[pos:pos+16])).upper()
        body=fvdata[pos+hdrsz:pos+size]
        secs=[]
        # walk file's sections
        p=0
        while p+4⩽len(body):
            ssize=u24(body[p:p+3]); stype=body[p+3]
            if ssize<4 or p+ssize>len(body): break
            secs.append((stype,body[p+4:p+ssize],p))
            p=(p+ssize+3)&~3
        out.append((g,fvdata[pos+18],secs))
        pos=(pos+size+7)&~7

def collect(fvmain_path):
    data=open(fvmain_path,'rb').read()
    fvs=[]
    # top-level section stream
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
    secs=[]
    parse_sections_raw(data,secs)
    for stype,payload,_ in [(s[0],s[1],None) for s in secs]:
        pass
    for s in secs:
        if s[0]==0x17:
            parse_fvs_raw(s[1],fvs)
    return fvs

if __name__=='__main__':
    import os
    fvs=collect(sys.argv[1])
    outdir=sys.argv[2]; os.makedirs(outdir,exist_ok=True)
    n=0
    for g,ftype,secs in fvs:
        for stype,body,p in secs:
            if stype in (0x10,0x12):  # PE32/TE
                fn=os.path.join(outdir,'%s_%d.pe'%(g.strip('{}'),n))
                open(fn,'wb').write(body)
                print(fn,len(body))
                n+=1
            elif stype==0x15:
                try: name=body.decode('utf-16-le').rstrip('\x00')
                except Exception: name=''
                open(os.path.join(outdir,'%s.ui'%g.strip('{}')),'w').write(name)
```

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```python
# decrypt.py
import uuid

data = open("spi_dump.bin", "rb").read()
g = uuid.UUID("4e2f1a33-8b5d-4c44-9e1b-63f0a2c7d901").bytes_le
i = data.find(g)  # 0x37e6c8 (старая ревизия) и 0x37f604 (новая)

ns = int.from_bytes(data[i - 8 : i - 4], "little")  # NameSize  = 22 (L"Ledger1147")
ds = int.from_bytes(data[i - 4 : i], "little")  # DataSize
blob = data[i + 16 + ns : i + 16 + ns + ds]  # 'TSUD' | len | 0x126 | payload
ct = blob[12:]

M = (1 << 64) - 1
KEY = bytes.fromhex("5ac3117e942bd8601fa945cc73e608bb")  # key_blob @ 0x10490


def crypt(buf):
    seed = int.from_bytes(KEY[8:16], "big")  # 0x1FA945CC73E608BB
    st = ((0xD1B54A32D192ED03 * len(buf)) & M) ^ seed
    st ^= st >> 27
    out = bytearray()
    for b in buf:
        t = st ^ (st >> 12)
        t = t ^ ((t << 25) & M)
        st = t ^ (t >> 27)
        out.append(b ^ (((st * 0x2545F4914F6CDD1D) & M) >> 56))
    return bytes(out)
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

02

ПРИМЕР РЕШЕНИЯ ЗАДАНИЯ

ПРОДОЛЖЕНИЕ

```
print(crypt(ct).decode())
```

Флаг

```
flag{1147_f1rst_r3c0rd_w4s_f0und}
```

ПЕРВАЯ ЗАПИСЬ
FORENSICS

ФЛАГ

```
flag{1147_f1rst_r3c0rd_w4s_f0und}
```

Задание «Первая запись» решено